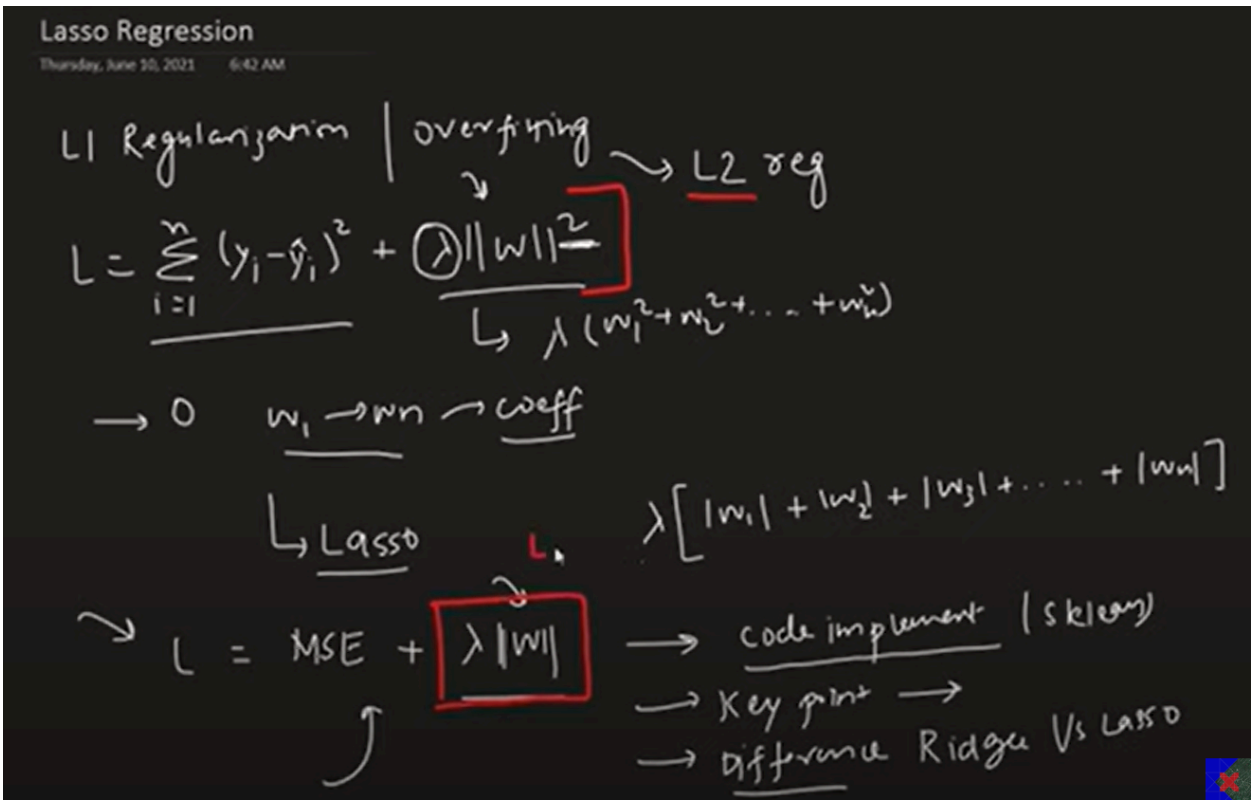
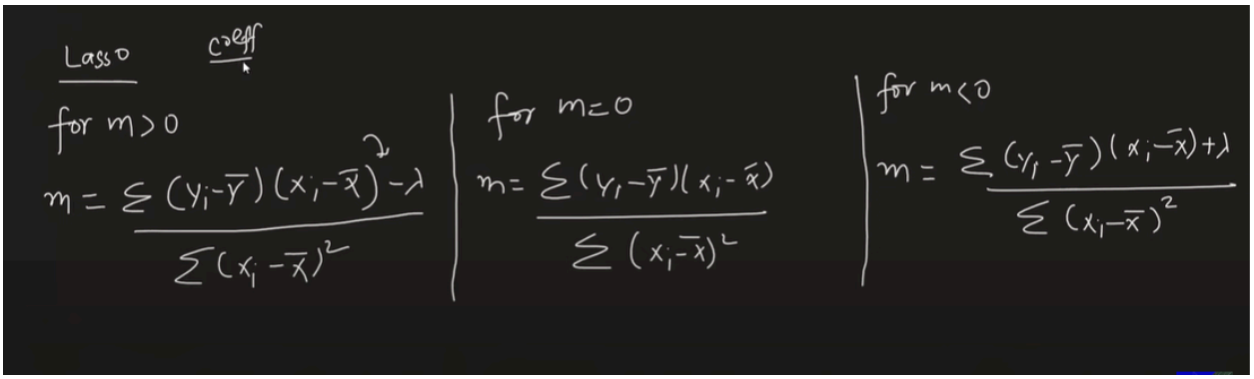
Lasso Regression

for Ridge L2 norm means all are squared w and in lasso its l1, means modulus of all w

Difference between Lasso and Ridge is that Lasso can do feature selection by making the the coeffient of some features zero it can exclude them, but Ridge never makes any coefficent zero, it can max get closer to zero but never absolute zero.





https://youtu.be/FN4aZPIAfI4?list=PLKnIA16_Rmvbr7zKYQuBfsVkjoLcJgxHH

```
# Import necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Lasso
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error
from sklearn.datasets import make_regression

# Generate synthetic data for regression
X, y = make_regression(n_samples=100, n_features=1, n_informa
tive=1, noise=20, random_state=13)

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, tes
t_size=0.2, random_state=42)

# Visualizing the dataset
plt.figure(figsize=(8, 5))
plt.scatter(X, y, color="blue", alpha=0.6, label="Data point
```

```
s")
plt.xlabel("Feature")
plt.ylabel("Target")
plt.title("Generated Regression Data")
plt.legend()
plt.show()

# Define alpha values for Lasso regression
alphas = [0, 1, 5, 10, 30]

# Train and visualize Lasso regression for different alpha va
lues
plt.figure(figsize=(12, 6))
plt.scatter(X, y, color="gray", alpha=0.5, label="Original Da
ta")

for alpha in alphas:
    lasso = Lasso(alpha=alpha)
    lasso.fit(X_train, y_train)
    y_pred = lasso.predict(X_test)

    # Plot regression line
    plt.plot(X_test, y_pred, label=f"Lasso α={alpha}")

plt.xlabel("Feature")
plt.ylabel("Target")
plt.title("Lasso Regression with Different Alpha Values")
plt.legend()
plt.show()

# Generate polynomial features for better modeling
m = 100
x1 = 5 * np.random.rand(m, 1) - 2
x2 = 0.7 * x1**2 - 2 * x1 + 3 + np.random.randn(m, 1)  # Non-
linear relation
```

```
# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(x1, x2, t
est_size=0.2, random_state=42)

# Define a polynomial regression model with Lasso regularizat
ion
lasso_pipeline = Pipeline([
    ("poly_features", PolynomialFeatures(degree=2, include_bi
as=False)),
    ("lasso_reg", Lasso(alpha=1.0))
])

# Train the model
lasso_pipeline.fit(X_train, y_train)

# Predict and evaluate
y_pred = lasso_pipeline.predict(X_test)
mse = mean_squared_error(y_test, y_pred)

print(f"Mean Squared Error (MSE) with Lasso Regression: {ms
e:.4f}")
```